

Week5_예습과제

[통계](#) [수정](#) [삭제](#)

카사바 · 방금 전

 0

Euron_2026-1



▼ 목록 보기

4/4



6장 합성곱 신경망II

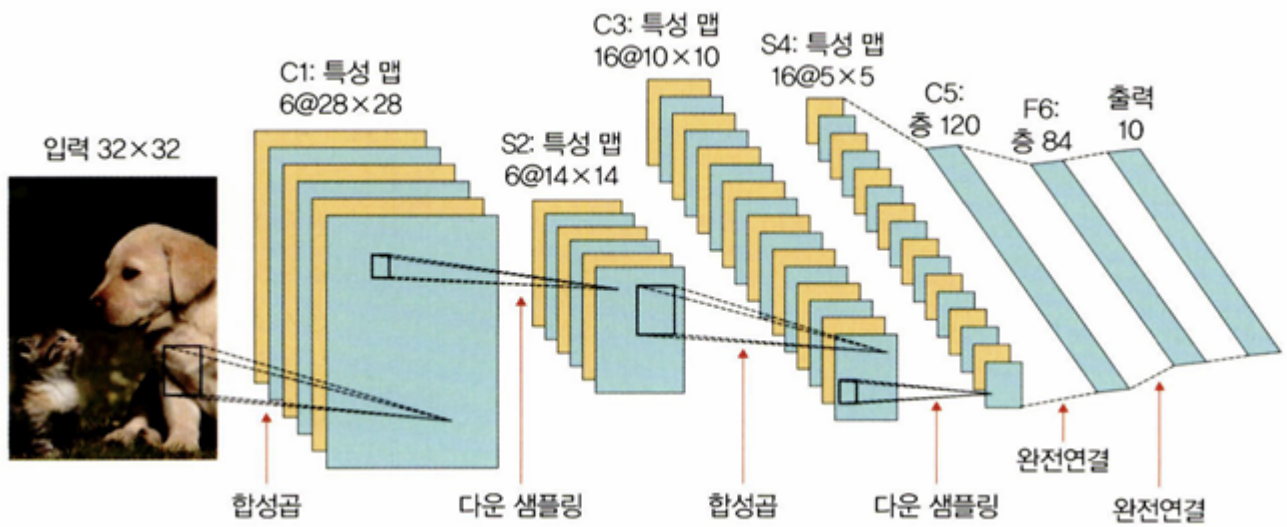
6.1 이미지 분류를 위한 신경망

입력 데이터로 이미지를 사용하는 분류(classification)는 특정 대상이 영상 내에 존재하는지 여부를 판단하는 것이다.

6.1.1 LeNet-5

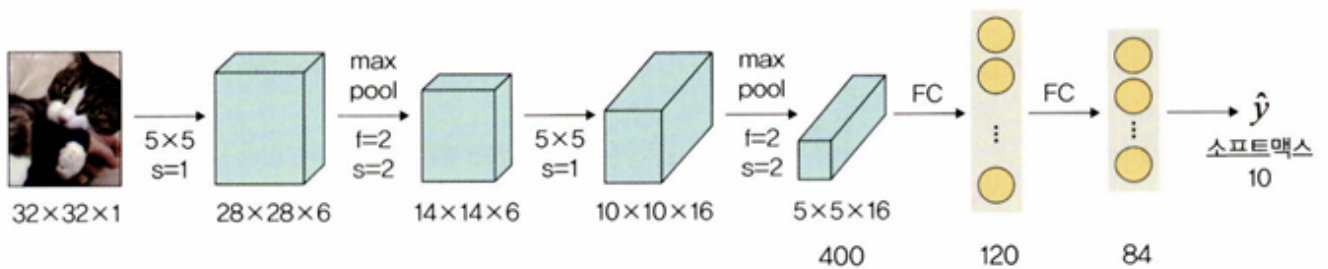
LeNet-5: 1995년 발표된 CNN의 초석

LeNet-5는 합성곱과 다운 샘플링(혹은 풀링)을 반복적으로 거치면서 마지막에 완전연결층에서 분류를 수행한다.



위 그림과 같이 합성곱 연산과 풀링을 반복하다 마지막 완전연결층으로 결과를 유닛(또는 노드)에 연결시킨다.

< LeNet-5 사용 예제 >



계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	32×32	—	—	—
합성곱층	6	28×28	5×5	1	렐루(ReLU)
최대 풀링층	6	14×14	2×2	2	—
합성곱층	16	10×10	5×5	1	렐루(ReLU)
최대 풀링층	16	5×5	2×2	2	—
완전연결층	—	120	—	—	렐루(ReLU)
완전연결층	—	84	—	—	렐루(ReLU)
완전연결층	—	2	—	—	소프트맥스(softmax)

tqdm: 진행 상태를 bar 형태로 가시화하여 보여 줌. 주로 모델 훈련에 대한 진행 상태를 확인하고자 할 때 사용

```
pip install --user tqdm
```

필요한 라이브러리 호출

```
import torch
import torchvision
from torch.utils.data import DataLoader, Dataset
from torchvision import transforms # 이미지 변환(전처리) 기능을 제공하는 라이브러리
from torch.autograd import Variable
from torch import optim # 경사 하강법을 이용하여 가중치를 구하기 위한 옵티마이저 라이브러리
import torch.nn as nn
import torch.nn.functional as F
import os # 파일 경로에 대한 함수들을 제공
import cv2
from PIL import Image
from tqdm import tqdm_notebook as tqdm # 진행 상황을 가시적으로 표현해 주는데, 특히 모델 학습 경과를
import random
from matplotlib import pyplot as pyplot

device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu") # 파이토치는 텐서플로와 C
```

이미지 데이터셋 전처리

```
class ImageTransform():
    def __init__(self, resize, mean, std):
        self.data_transform = {
            'train': transforms.Compose([
                transforms.RandomResizedCrop(resize, scale=(0.5, 1.0)),
```

```

        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize(mean, std)
    ]), # ①
    'val': transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(resize),
        transforms.ToTensor(),
        transforms.Normalize(mean, std)
    ])
}

def __call__(self, img, phase): # ②
    return self.data_transform[phase](img)

```

① 토치비전 라이브러리를 이용하면 이미지 전처리를 쉽게 할 수 있다.

```

train_transforms = transforms.Compose(
    (a)
    [ transforms.RandomResizedCrop(resize, scale=(0.5,1.0)),
      (b)
      transforms.RandomHorizontalFlip(),
      (c)
      transforms.ToTensor(), transforms.Normalize(mean, std)] )
    (d)                      (e)

```

① transforms.Compose: 이미지를 변형할 수 있는 방식들의 묶음

② transforms.RandomResizedCrop: 입력 이미지를 주어진 크기(resize: 224 x 224)로 조정한다. scale은 원래 이미지를 임의의 크기(0.5~1.0(50~100%))만큼 면적을 무작위로 자르겠다는 의미

③ transforms.RandomHorizontalFlip: 주어진 확률로 이미지를 수평 반전시킨다. 이때 확률 값을 지정하지 않으면 기본값인 0.5의 확률로 이미지들이 수평 반전된다.

④ transforms.ToTensor: ImageFolder 메서드를 비롯해서 torchvision 메서드는 이미지를 읽을 때 파이썬 이미지 라이브러리인 PIL을 사용한다. PIL을 사용해서 이미지를 읽으면 생성되는 이미지는 범위가 [0, 255]이며, 배열의 차원이 (높이 H x 너비 W x 채널 수 C)로 표현된다. 이후 효율적인 연산을 위해 torch.FloatTensor 배열로 바꾸어야 하는데, 이때 픽셀 값의 범위는 [0.0, 1.0] 사이가 되고 차원의 순서도 (채널 수 C x 높이 H x 너비 W)로 바뀐다. 이러한 작업을 수행해주는 메서드가 ToTensor()이다.

⑤ transforms.Normalize: 전이 학습에서 사용하는 사전 훈련된 모델들은 대개 ImageNet 데이터셋에서 훈련되었다. 따라서 사전 훈련된 모델을 사용하기 위해서는 ImageNet 데이터의 각 채널별 평균과 표준편차에 맞는 정규화를 해 주어야 한다. Normalize 메서드 안에 사용된 (mean: 0.485, 0.456, 0.406), (std: 0.229, 0.224, 0.225)는 ImageNet에서 이미지들의 RGB

채널마다의 평균과 표준편차를 의미한다. OpenCV를 사용해서 이미지를 읽어 온다면 RGB가 아닌 BGR이므로 채널 순서에 주의해야 한다.

② **call** 함수는 클래스를 호출할 수 있도록 하는 메서드이다. 즉, 클래스에 **call** 함수가 있을 경우 클래스 객체 자체를 호출하면 **call** 함수의 리턴 값이 반환된다.

훈련용으로 400개의 이미지, 검증용으로 92개의 이미지, 테스트용으로 10개의 이미지가 사용된다.

이미지 데이터셋을 불러온 후 훈련, 검증, 테스트로 분리

```
cat_directory = r"/content/drive/MyDrive/data/dogs-vs-cats/Cat/"
dog_directory = r"/content/drive/MyDrive/data/dogs-vs-cats/Dog/"

cat_images_filepaths = sorted([os.path.join(cat_directory, f) for f in
                                os.listdir(cat_directory)]) # ①
dog_images_filepaths = sorted([os.path.join(dog_directory, f) for f in
                                os.listdir(dog_directory)])
images_filepaths = [*cat_images_filepaths, *dog_images_filepaths] # 개와 고양이 이미지들을 합쳐서
correct_images_filepaths = [i for i in images_filepaths if cv2.imread(i) is not None] # ②

random.seed(42) # ③
random.shuffle(correct_images_filepaths)
train_images_filepaths = correct_images_filepaths[:400] # 훈련용 400개의 이미지
val_images_filepaths = correct_images_filepaths[400:-10] # 검증용 92개의 이미지
test_images_filepaths = correct_images_filepaths[-10:] # 테스트용 10개의 이미지
print(len(train_images_filepaths), len(val_images_filepaths), len(test_images_filepaths))
```

① 고양이 이미지 데이터를 가져온다.

```
cat_images_filepaths = sorted([os.path.join(cat_directory, f)
                                for f in os.listdir(cat_directory)])
```

(a) (b) (c)

① sorted: 데이터를 정렬된 리스트로 만들어서 반환한다.

② os.path.join: 경로와 파일명을 결합하거나 분할된 경로를 하나로 합치고 싶을 때 사용한다.

③ os.listdir: 지정한 디렉터리 내 모든 파일의 리스트를 반환한다. Cat 디렉터리의 이미지 파일들을 모두 반환한다.

② images_filepaths에서 이미지 파일들을 불러온다.

```
correct_images_filepaths = [i for i in images_filepaths
                             if cv2.imread(i) is not None]
```

(a)
(b)
(c)

- ① for 반복문을 이용해 가져온 데이터에 대해 i를 이용하여 리스트로 만든다.
 - ② 반복문을 이용해 images_filepaths에서 이미지 데이터를 검색한다.
 - ③ 조건문. cv2.imread() 함수를 이용하여 모든 이미지 데이터를 읽어 온다. (not None 상태, 즉 더 이상 데이터를 찾을 수 없을 때까지)
- ③ 넘파이 random() 함수는 임의의 난수를 생성하는데, 이때 난수를 생성하기 위해 사용되는 것이 시드 값이다. 또 Numpy.random.seed() 메서드는 상태를 초기화한다. 즉, 이 모듈이 호출될 때마다 임의의 난수가 재생성된다. 하지만 특정 시드 값을 부여하면 상태가 저장되기 때문에 동일한 난수를 생성한다.

테스트 데이터셋 이미지 확인 함수

```
def display_image_grid(images_filepaths, predicted_labels=(), cols=5):
    rows = len(images_filepaths) // cols
    figure, ax = pyplot.subplots(nrows=rows, ncols=cols, figsize=(12, 6))
    for i, image_filepath in enumerate(images_filepaths):
        image = cv2.imread(image_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # ①
        true_label = os.path.normpath(image_filepath).split(os.sep)[-2] # ②
        predicted_label = predicted_labels[i] if predicted_labels else true_label # ③
        color = "green" if true_label == predicted_label else "red" # 예측과 정답(레이블)이 동일하면 초
        ax.ravel()[i].imshow(image) # 개별 이미지 출력
        ax.ravel()[i].set_title(predicted_label, color=color) # predicted_label을 타이틀로 사용
        ax.ravel()[i].set_axis_off() # 이미지 축 제거
    plt.tight_layout() # 이미지 여백 조정
    plt.show()
```

- ① cv2.cvtColor는 이미지의 색상을 변경하기 위해 사용된다.

```
cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

(a)
(b)

- ① 첫 번째 파라미터: 입력 이미지

⑥ 두 번째 파라미터: 변환할 이미지의 색상을 지정. 이 코드에서는 BGR 채널 이미지를 RGB로 변경하겠다는 의미이다.

② 이미지의 전체 경로를 정규화하고 분할한다.

```
true_label = os.path.normpath(image_filepath).split(os.sep)[-2]
```

(a) (b)

① os.path.normpath: 경로명을 정규화한다. 예를 들어 모두 다르게 생긴 경로를 A/B 형식으로 통일한다.

② split(os.sep): 경로를 / 혹은 \ 기준으로 분할할 때 사용한다. 또, split(os.sep)[-2]를 적용하면 뒤에서 두 번째 값을 반환한다.

③

```
predicted_label = predicted_labels[i] if predicted_labels else true_label
```

(a) (b)

① predicted_labels 값이 있으면 그 값을 predicted_label로 사용한다.

② predicted_labels 값이 없으면 true_label 값을 predicted_label로 사용한다.

테스트 데이터셋 이미지 출력

```
display_image_grid(test_images_filepaths)
```



다음으로 데이터셋에 학습할 데이터의 경로를 정의하고 그 경로에서 데이터를 읽어 온다. 데이터셋 크기가 클 수 있으므로 `__init__` 에서 전체 데이터를 읽어 오는 것이 아니라 경로만 저장해 놓고, `__getitem__` 메서드에서 이미지를 읽어 온다. 즉, 데이터를 어디에서 가져올지 결정한다. 이후 데이터로더에서 데이터셋의 데이터를 메모리로 불러오는데, 한꺼번에 전체 데이터를 불러오는 것이 아니라 배치 크기만큼 분할하여 가져온다.

`DogvsCatDataset()` 클래스는 데이터를 불러오는 방법을 정의한다. 예제의 목적은 다수의 개와 고양이 이미지가 포함된 데이터에서 이들을 예측하는 것이다. 따라서 레이블(정답) 이미지에서 고양이와 개가 포함될 확률을 코드로 구현한다. 고양이의 레이블은 0, 개의 레이블은 1이 된다.

이미지 데이터셋 클래스 정의

```
class DogvsCatDataset(Dataset):
    def __init__(self, file_list, transform=None, phase='train'): # 데이터셋의 전처리(데이터 변형 적용)
        self.file_list = file_list
        self.transform = transform # DogvsCatDataset 클래스를 호출할 때 transform에 대한 매개변수를 받아
        self.phase = phase # 'train' 적용

    def __len__(self): # images_filepaths 데이터셋의 전체 길이를 반환
        return len(self.file_list)

    def __getitem__(self, idx): # 데이터셋에서 데이터를 가져오는 부분으로 결과는 텐서 형태가 됨
        img_path = self.file_list[idx]
        img = Image.open(img_path) # img_path 위치에서 이미지 데이터들을 가져옴
        img_transformed = self.transform(img, self.phase) # 이미지에 'train' 전처리를 적용
        label = img_path.split('/')[-1].split('.')[0] # ①
        if label == 'dog':
            label = 1
        elif label == 'cat':
```

```
label = 0
return img_transformed, label
```

① 이미지 데이터에 대한 레이블 값(dog, cat)을 가져온다.

```
label = img_path.split('/')[-1].split('.')[0]
```

(a) (b) (c)

① img_path: 이미지가 위치한 전체 경로를 보여 준다. 이때 이미지 데이터의 레이블(dog)을 가져오려면 '/'와 '.'을 제거해야 한다. 다음의 ②와 ③이 '/'와 '.'을 제거하기 위해 사용된다.

② img_path.split('/')[-1]: 이미지의 전체 경로에서 '/'를 제거한다.

③ split('.')[0]: 'dog.113.jpg'에서 '.'을 제거한다. '.'을 기준으로 분리시켰고, 분리된 값들에서 첫번째 값([0])을 가져오면 결과는 다음과 같다.

dog

전처리에서 사용할 변수에 대한 값을 정의한다.

변수 값 정의

```
size = 224
mean = (0.485, 0.456, 0.406)
std = (0.229, 0.224, 0.225)
batch_size = 32
```

훈련과 검증 용도의 데이터셋을 정의한다. 앞에서 정의한 DogvsCatDataset() 클래스를 이용하여 훈련과 검증 데이터셋을 준비하되 전처리도 함께 적용하도록 한다.

이미지 데이터셋 정의

```
train_dataset = DogvsCatDataset(train_images_filepaths, transform=ImageTransform(size, mean, s
val_dataset = DogvsCatDataset(val_images_filepaths, transform=ImageTransform(size, mean, std),

index = 0
print(train_dataset.__getitem__(index)[0].size()) # 훈련 데이터(train_dataset.__getitem__[0][0])
print(train_dataset.__getitem__(index)[1]) # 훈련 데이터의 레이블 출력
```

다음은 훈련 데이터의 크기와 레이블에 대한 출력 결과이다.

```
torch.Size([3, 224, 224])
0
```

이미지는 컬러 상태에서 224x224 크기를 가지며 레이블이 0으로 출력되었다. 즉, 훈련 데이터셋의 레이블 (`train_dataset.__getitem__(index)[0]`) 이 0 값을 갖기 때문에 고양이 이미지가 포함되어 있다는 것을 유추할 수 있다. 전처리와 함께 데이터셋을 정의했기 때문에 이제 메모리로 불러와서 훈련을 위한 준비를 한다.

데이터로더 정의

```
train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True) # ①
val_dataloader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
dataloader_dict = {'train': train_dataloader, 'val': val_dataloader} # 훈련 데이터셋과 검증 데이터

batch_iterator = iter(train_dataloader)
inputs, label = next(batch_iterator)
print(inputs.size())
print(label)
```

① 파이토치의 데이터로더는 배치 관리를 담당한다. 한 번에 모든 데이터를 불러오면 메모리에 부담을 줄 수 있기 때문에 데이터를 그룹으로 쪼개서 조금씩 불러온다.

```
train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
                             ①                ②                ③
```

- ① 첫 번째 파라미터: 데이터를 불러오기 위한 데이터셋
- ② `batch_size`: 한 번에 메모리로 불러올 데이터 크기로, 여기에서는 32개씩 데이터를 가져온다.
- ③ `shuffle`: 메모리로 데이터를 가져올 때 임의로 섞어서 가져오도록 한다.

다음으로 데이터셋을 학습시킬 모델의 네트워크를 설계하기 위한 클래스를 생성한다.

모델의 네트워크 클래스

```
class LeNet(nn.Module):
    def __init__(self):
        super(LeNet, self).__init__()

        self.cnn1 = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=5, stride=1, padding=0)

        self.relu1 = nn.ReLU() # ReLU 활성화 함수
```

```

self.maxpool1 = nn.MaxPool2d(kernel_size=2) # 최대 풀링이 적용된다. 적용 이후 출력 형태는 220/220
self.cnn2 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size=5, stride=1, padding=0)
self.relu2 = nn.ReLU()
self.maxpool2 = nn.MaxPool2d(kernel_size=2) # 최대 풀링이 적용된다. 적용 이후 출력 형태는 (32,53

self.fc1 = nn.Linear(32*53*53, 512)
self.relu5 = nn.ReLU()
self.fc2 = nn.Linear(512,2)
self.output = nn.Softmax(dim=1)

def forward(self, x):
    out = self.cnn1(x)
    out = self.relu1(out)
    out = self.maxpool1(out)
    out = self.cnn2(out)
    out = self.relu2(out)
    out = self.maxpool2(out)
    out = out.view(out.size(0),-1) # 완전연결층에 데이터를 전달하기 위해 데이터 형태를 1차원으로 바꾼다.
    out = self.fc1(out)
    out = self.fc2(out)
    out = self.output(out)
    return out

```

네트워크의 각 부분들을 통과할 때마다 입력과 출력의 형태가 바뀌는데, 그 계산은 다음 수식과 같다.

Conv2d 계층에서의 출력 크기 구하는 공식

$$\text{출력 크기} = (W - F + 2P) / S + 1$$

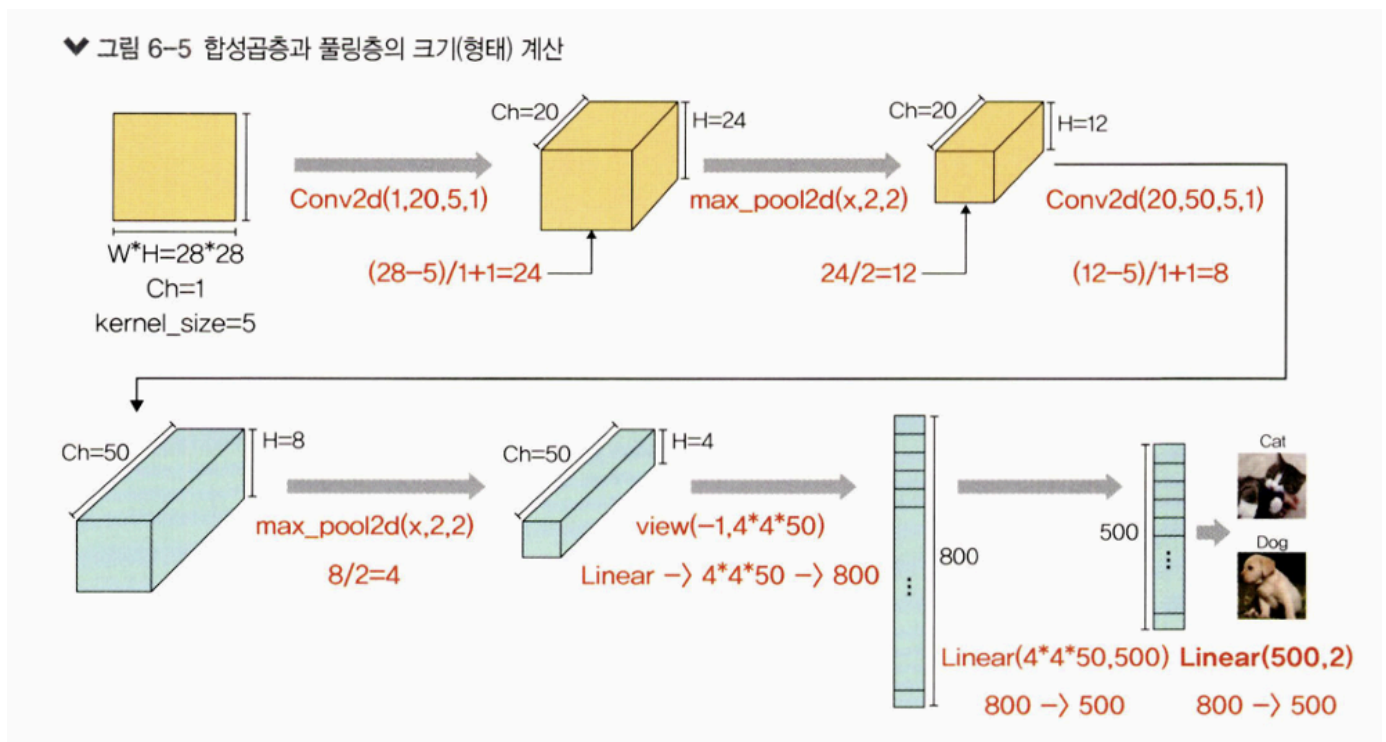
- W : 입력 데이터의 크기(input_volume_size)
- F : 커널 크기(kernel_size)
- P : 패딩 크기(padding_size)
- S : 스트라이드(strides)

MaxPool2d 계층에서의 출력 크기 구하는 공식

출력 크기 = IF/F

- IF : 입력 필터의 크기(input_filter_size, 또한 바로 앞의 Conv2d의 출력 크기이기도 합니다)
- F : 커널 크기(kernel_size)

각 계층에 대한 결과는 다음과 같다.



LeNet()을 model이라는 이름으로 객체 생성하여 모델 학습을 위한 준비를 한다.

모델 객체 생성

```
model = LeNet()
print(model)
```

다음은 생성한 model에 대한 출력 결과이다.

```

LeNet(
  (cnn1): Conv2d(3, 16, kernel_size=(5, 5), stride=(1, 1))
  (relu1): ReLU()
  (maxpool1): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (cnn2): Conv2d(16, 32, kernel_size=(5, 5), stride=(1, 1))
  (relu2): ReLU()
  (maxpool2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc1): Linear(in_features=89888, out_features=512, bias=True)
  (relu5): ReLU()
  (fc2): Linear(in_features=512, out_features=2, bias=True)
  (output): Softmax(dim=1)
)

```

출력 결과가 한눈에 들어오지 않는다면 torchsummary 라이브러리를 사용할 수 있다.
torchsummary는 케라스와 같은 형태로 모델을 출력해 볼 수 있는 라이브러리다.

다음 명령을 실행하여 torchsummary 라이브러리를 설치한다.

```
pip install torchsummary
```

설치 완료 후 torchsummary 라이브러리를 사용해 모델의 네트워크 구조를 다시 확인한다.

torchsummary 라이브러리를 이용한 모델의 네트워크 구조 확인

```

from torchsummary import summary
summary(model, input_size=(3,224,224)) # ①

```

① summary를 통해 모델의 네트워크 관련 정보를 확인할 수 있으며 여기에서 사용되는 파라미터는 다음과 같다.

`summary(model, input_size=(3,224,224))`
①
②

① 첫 번째 파라미터: 모델의 네트워크

② 두 번째 파라미터: 입력 값으로 (채널, 너비, 높이)를 사용한다. 여기서는 (3,224,224)를 입력으로 지정한다.

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 16, 220, 220]	1,216
ReLU-2	[-1, 16, 220, 220]	0
MaxPool2d-3	[-1, 16, 110, 110]	0
Conv2d-4	[-1, 32, 106, 106]	12,832
ReLU-5	[-1, 32, 106, 106]	0
MaxPool2d-6	[-1, 32, 53, 53]	0
Linear-7	[-1, 512]	46,023,168
Linear-8	[-1, 2]	1,026
Softmax-9	[-1, 2]	0
Total params: 46,038,242		
Trainable params: 46,038,242		
Non-trainable params: 0		
Input size (MB): 0.57		
Forward/backward pass size (MB): 19.47		
Params size (MB): 175.62		
Estimated Total Size (MB): 195.67		

출력되는 정보로는 총 파라미터 수, 입력 크기, 네트워크의 총 크기 등이 있다.

모델의 학습 가능한 파라미터 수를 `model.parameters()`를 이용하여 확인한다.

학습 가능한 파라미터 수 확인

```
def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

print(f'The model has {count_parameters(model):,} trainable parameters')
```

```
The model has 46,038,242 trainable parameters
```

파라미터가 꽤 많기 때문에 캐글에서 내려받은 이미지의 일부만 사용하여 학습 시간을 줄인다.

옵티마이저와 손실 함수 정의

```
optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9) # ㉑
criterion = nn.CrossEntropyLoss()
```

① 경사 하강법으로 모멘텀 SGD를 사용한다. 모멘텀 SGD는 SGD에 관성이 추가된 것으로 매번 기울기를 구하지만 가중치를 수정하기 전에 이전 수정 방향(+, -)을 참고하여 같은 방향으로 일정한 비율만 수정되게 하는 방법이다. 사용되는 파라미터는 다음과 같다.

`optim.SGD(model.parameters(), lr=0.001, momentum=0.9)`

(a) (b) (c)

① 첫 번째 파라미터: 경사 하강법을 통해 궁극적으로 업데이트하고자 하는 파라미터는 가중치와 바이어스이다.

② lr(learning rate): 가중치를 변경할 때 얼마나 크게 변경할지 결정한다.

③ momentum: SGD를 적절한 방향으로 가속화하며 흔들림(진동)을 줄여 주는 매개변수이다.

모델의 파라미터와 손실 함수를 CPU에 할당

```
model = model.to(device)
criterion = criterion.to(device)
```

모델 학습 함수 정의

```
from IPython.core.interactiveshell import SingletonConfigurable
def train_model(model, dataloader_dict, criterion, optimizer, num_epoch):
    since = time.time()
    best_acc = 0.0

    for epoch in range(num_epoch): # epoch을 10으로 설정했으므로 10회 반복
        print('Epoch {}/{}'.format(epoch+1, num_epoch))
        print('-'*20)

        for phase in ['train', 'val']:
            if phase == 'train':
                model.train() # 모델을 학습시키겠다는 의미
            else:
                model.eval()

            epoch_loss = 0.0
            epoch_corrects = 0

            for inputs, labels in tqdm(dataloader_dict[phase]): # dataloader_dict는 훈련 데이터셋(train)
                inputs = inputs.to(device) # 훈련 데이터셋을 CPU에 할당
                labels = labels.to(device)
                optimizer.zero_grad() # 역전파 단계를 실행하기 전에 기울기를 0으로 초기화

                with torch.set_grad_enabled(phase == 'train'):
                    outputs = model(inputs)
                    _, preds = torch.max(outputs, 1)
```

```

loss = criterion(outputs, labels) # 손실함수를 이용한 오차 계산

if phase == 'train':
    loss.backward() # 모델의 학습 가능한 모든 파라미터에 대해 기울기 계산
    optimizer.step() # optimizer의 step 함수 호출해 파라미터 갱신

    epoch_loss += loss.item() * inputs.size(0) # ①
    epoch_corrects += torch.sum(preds == labels.data) # 정답과 예측이 일치하면 그것의 합계를

epoch_loss = epoch_loss / len(dataloader_dict[phase].dataset) # 최종 오차 계산(초라를 데이터
epoch_acc = epoch_corrects.double() / len(dataloader_dict[phase].dataset) # 최종 정확도(e

print('{} Loss: {:.4f} Acc: {:.4f}'.format(phase, epoch_loss, epoch_acc))

if phase == 'val' and epoch_acc > best_acc: # 검증 데이터셋에 대한 가장 최적의 정확도를 저장
    best_acc = epoch_acc
    best_model_wts = model.state_dict()

time_elapsed = time.time() - since
print('Training complete in {:.0f}m {:.0f}s'.format(
    time_elapsed // 60, time_elapsed % 60))
print('Best val Acc: {:.4f}'.format(best_acc))
return model

```

①

```

criterion = nn.CrossEntropyLoss(reduction='mean')

```

reduction 파라미터의 기본값은 'mean'이다. 'mean'은 정답과 예측 값의 오차를 구한 후 그 값들의 평균을 반환한다. 즉, 전체 오차를 배치 크기로 나눔으로써 평균을 반환하기 때문에 epoch_loss를 계산하는 동안 loss.item()과 inputs.size(0)을 곱해 준다.

모델을 학습시키기 위해 train_model을 호출한다.

모델 학습

```

import time

num_epoch = 10
model = train_model(model, dataloader_dict, criterion, optimizer, num_epoch)

```

```

-----
100% ██████████ 13/13 [00:28<00:00, 1.99s/it]
train Loss: 0.6186 Acc: 0.6650
100% ██████████ 3/3 [00:03<00:00, 1.11s/it]
val Loss: 0.6949 Acc: 0.5761
Epoch 7/10
-----
100% ██████████ 13/13 [00:29<00:00, 2.12s/it]
train Loss: 0.6141 Acc: 0.6700
100% ██████████ 3/3 [00:02<00:00, 1.05it/s]
val Loss: 0.7086 Acc: 0.5652
Epoch 8/10
-----
100% ██████████ 13/13 [00:29<00:00, 2.02s/it]
train Loss: 0.6346 Acc: 0.6500
100% ██████████ 3/3 [00:02<00:00, 1.07it/s]
val Loss: 0.6843 Acc: 0.5435
Epoch 9/10
-----
100% ██████████ 13/13 [00:29<00:00, 1.97s/it]
train Loss: 0.6239 Acc: 0.6925
100% ██████████ 3/3 [00:02<00:00, 1.06it/s]
val Loss: 0.6922 Acc: 0.5543
Epoch 10/10
-----
100% ██████████ 13/13 [00:28<00:00, 1.87s/it]
train Loss: 0.6216 Acc: 0.6600
100% ██████████ 3/3 [00:03<00:00, 1.01s/it]
val Loss: 0.6718 Acc: 0.5978
Training complete in 5m 33s
Best val Acc: 0.597826

```

테스트 데이터셋을 모델에 적용해 정확도를 측정할 것이다. 측정 결과는 데이터 프레임에 담은 후 CSV 파일로 저장한다. 테스트 용도의 데이터셋을 이용하므로 `model.eval()`을 사용한다.

모델 테스트를 위한 함수 정의

```
import pandas as pd
```

```
id_list = []
pred_list = []
```

```

_id = 0
with torch.no_grad(): # 역전파 중 텐서에 대한 변화도를 계산할 필요가 없음을 나타내는 것으로, 훈련 데이터셋
    for test_path in tqdm(test_images_filepaths): # 테스트 데이터셋 이용
        img = Image.open(test_path)
        _id = test_path.split('/')[-1].split('.')[1]
        transform = ImageTransform(size, mean, std)
        img = transform(img, phase='val') # 테스트 데이터셋 전처리 적용
        img = img.unsqueeze(0) # ①
        img = img.to(device)

        model.eval()
        outputs = model(img)
        preds = F.softmax(outputs, dim=1)[: ,1].tolist() # ②
        id_list.append(_id)
        pred_list.append(preds[0])

res = pd.DataFrame({'id':id_list, 'label':pred_list}) # 테스트 데이터셋의 예측 결과인 id와 레이블을 저장

res.sort_values(by='id', inplace=True)
res.reset_index(drop=True, inplace=True)

res.to_csv('/content/drive/MyDrive/data/LeNet', index=False) # 데이터 프레임을 CSV 파일로 저장

```

① torch.unsqueeze는 텐서에 차원을 추가할 때 사용한다. (0)은 차원이 추가될 위치를 의미한다.

예를 들어 형태가 (3)인 텐서가 있다. 0 위치에 차원을 추가하면 형태가 (1,3)이 된다. 즉 행 한 개와 열 세 개의 구조를 갖는 텐서가 만들어진다.

2D에 텐서가 추가된다면:

- 형태가 (2,2)인 2D 텐서가 있을 때 0 위치에 차원을 추가하면 텐서 모양이 (1,2,2)가된다. 이는 하나의 채널, 행 두 개와 열 두 개를 의미한다.
- 1 위치에 차원을 추가하면 (2,1,2) 즉 채널 두 개, 행 한 개, 열 두 개가 된다.
- 2 위치에 차원을 추가하면 (2,2,1) 즉 채널 두 개, 행 두 개, 열 한 개가 된다.

② 소프트맥스는 지정된 차원을 따라 텐서의 요소(텐서의 개별 값)가 (0,1) 범위에 있고 합계가 1이 되도록 크기를 다시 조정한다.

$$F.softmax(\underbrace{outputs}_{(a)}, \underbrace{dim=1}_{(b)})[\underbrace{: , 1}_{(c)}].tolist()$$

② outputs에 softmax를 적용해 각 행의 합이 1이 되도록 한다.

⑥ ⑤의 값 중 모든 행(:)에서 두 번째 칼럼(1번째 인덱스)을 가져온다.

⑦ 배열을 리스트 형태로 변환한다.

100%

10/10 [00:00<00:00, 21.98it/s]

이는 모델 예측 함수를 실행한 결과이다. 예측 결과를 CSV 파일로 저장하는 함수이다.

테스트 데이터셋을 모델에 적용한 결과를 LeNet 파일로 저장해 두었다.

테스트 데이터셋의 예측 결과 호출

```
res.head(10)
```

	id	label
0	109	0.288891
1	145	0.286602
2	15	0.544323
3	162	0.369177
4	167	0.506618
5	200	0.290488
6	210	0.560790
7	211	0.446696
8	213	0.296178
9	224	0.622251

예측 결과 레이블이 0.5보다 크면 개를 의미하고, 0.5보다 작으면 고양이를 의미한다.

예측 결과를 시각적으로 표현하기 위한 함수를 정의한다.

테스트 데이터셋 이미지를 출력하기 위한 함수 정의

```
class_ = classes = {0:'cat', 1:'dog'} # 개와 고양이에 대한 클래스 정의
def display_image_grid(images_filepaths, predicted_labels=(), cols=5):
```



```

rows = len(images_filepaths) // cols
figure, ax = plt.subplots(nrows=rows, ncols=cols, figsize=(12, 6))
for i, image_filepath in enumerate(images_filepaths):
    image = cv2.imread(image_filepath)
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
    a = random.choice(res['id'].values) # 데이터 프레임의 id라는 컬럼에서 임의로 데이터를 가져옴
    label = res.col[res['id'] == a, 'label'].values[0]
    if label > 0.5: # 레이블 값이 0.5보다 크다면 개
        label = 1
    else: # 레이블 값이 0.5보다 작으면 고양이
        label = 0
    ax.ravel()[i].imshow(image)
    ax.ravel()[i].set_title(class_[label])
    ax.ravel()[i].set_axis_off()
plt.tight_layout()
plt.show()

```

테스트 데이터셋에 대한 예측 결과를 시각적으로 보여 주기 위해 앞에서 정의한 함수를 호출한다. 이때 전달되는 파라미터는 테스트 데이터셋이다.

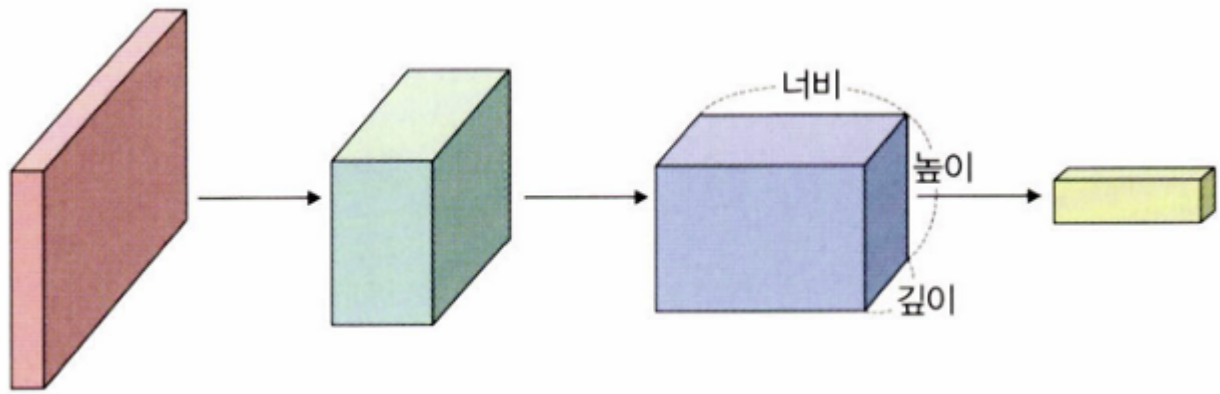
테스트 데이터셋 예측 결과 이미지 출력

```
display_image_grid(test_images_filepaths)
```

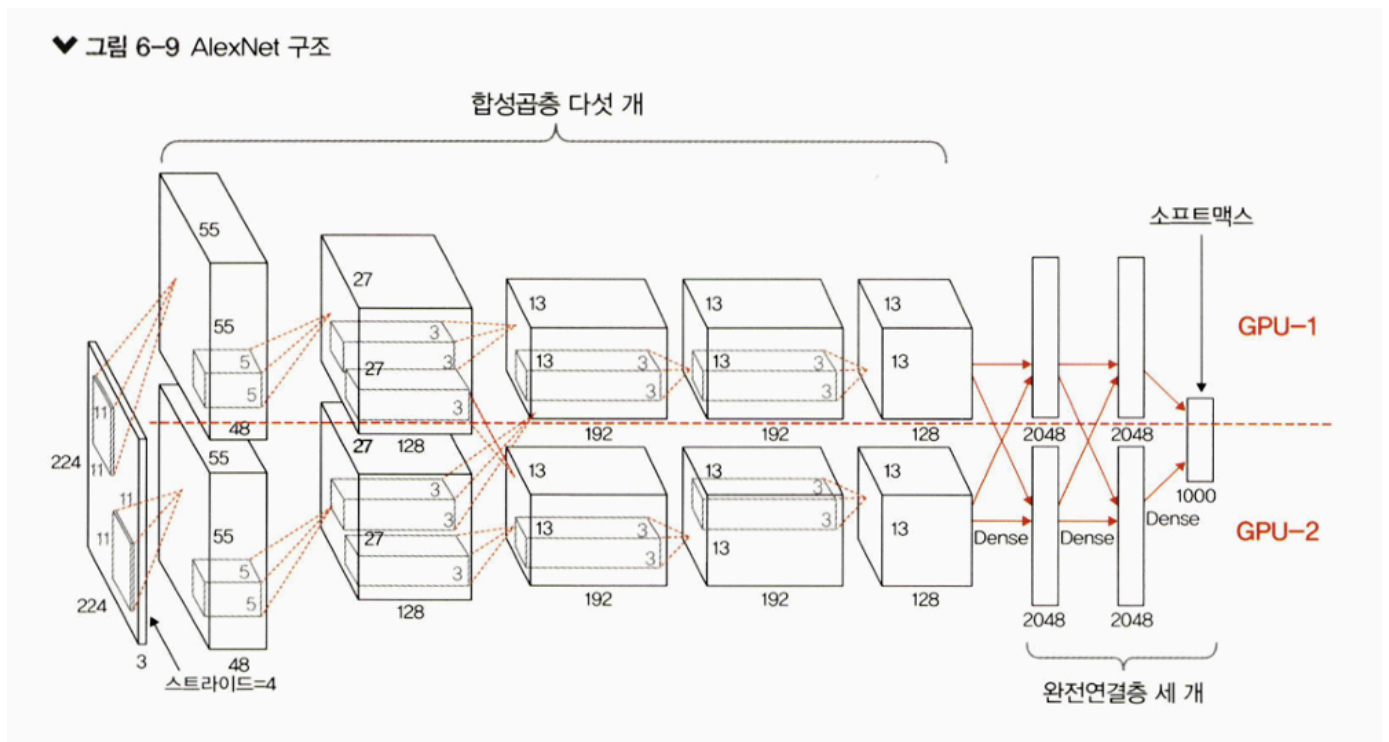


6.1.2 AlexNet

AlexNet은 ImageNet 영상 데이터베이스를 기반으로 한 화상 인식 대회인 'ILSVRC 2012'에서 우승한 CNN 구조이다.



이미지의 깊이는 보통 RGB 세 성분에 의해 3으로 시작하지만, 합성곱을 거치며 특성 맵이 만들어지고 이에 따라 중간 영상의 깊이가 달라진다.



AlexNet은 합성곱층 다섯 개와 완전연결층 세 개로 구성되어 있으며, 맨 마지막 완전연결층은 카테고리 1000개를 분류하기 위해 소프트맥스 활성화 함수를 사용한다. GPU 두 개를 기반으로 한 병렬 구조인 점을 제외하면 LeNet-5와 크게 다르지 않다.

AlexNet의 합성곱층에서 사용된 활성화 함수는 ReLU이다.

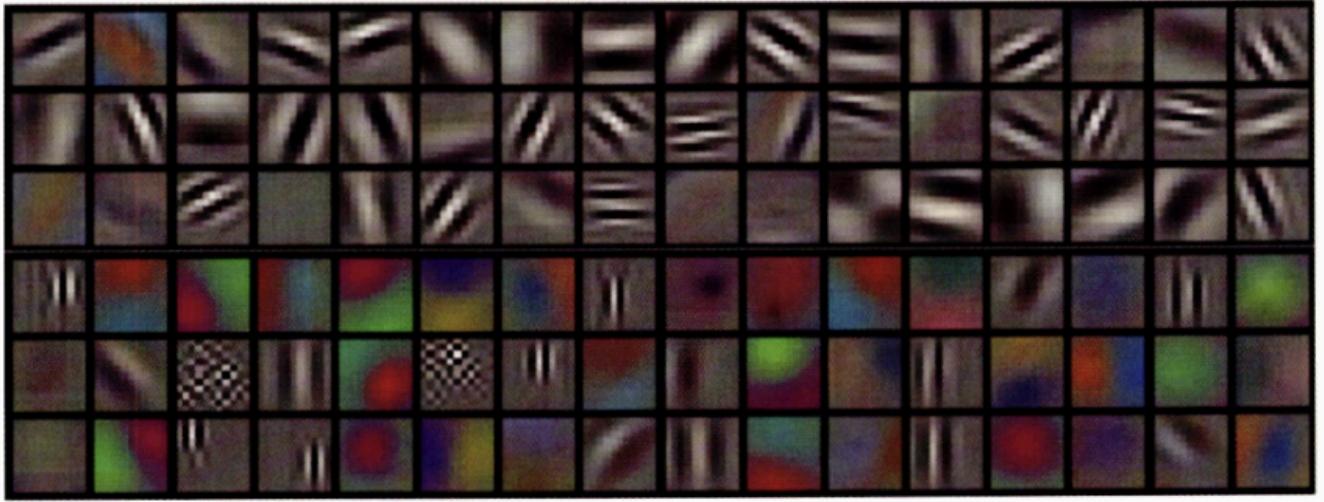
♥ 표 6-2 AlexNet 구조 상세

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	227×227	—	—	—
합성곱층	96	55×55	11×11	4	렐루(ReLU)
최대 풀링층	96	27×27	3×3	2	—
합성곱층	256	27×27	5×5	1	렐루(ReLU)
최대 풀링층	256	13×13	3×3	2	—
합성곱층	384	13×13	3×3	1	렐루(ReLU)
합성곱층	384	13×13	3×3	1	렐루(ReLU)
합성곱층	256	13×13	3×3	1	렐루(ReLU)
최대 풀링층	256	6×6	3×3	2	—
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	1000	—	—	소프트맥스(softmax)

네트워크에는 학습 가능한 변수가 총 6600만 개 있다. 네트워크에 대한 입력은 227x227x3 크기의 RGB 이미지이며, 각 클래스에 해당하는 1000x1 확률 벡터를 출력한다.

AlexNet의 첫 번째 합성곱층 커널 크기는 11x11x3이며, 스트라이드를 4로 적용하여 특성맵을 96개 출력하기 때문에 55x55x96의 출력을 갖는다. 첫 번째 계층을 거치면서 GPU-1에서는 주로 컬러와 상관없는 정보를 추출하기 위한 커널이 학습되고, GPU-2에서는 주로 컬러와 관련된 정보를 추출하기 위한 커널이 학습된다.

각 GPU 결과가 다음 그림과 같이 나타난다.



필요한 라이브러리 호출

:
 def __init__(self, resize, mean, std):
 self.data_transform = {
 'train': transforms.Compose([
 transforms.RandomResizedCrop(resize, scale=(0.5, 1.0)),
 transforms.RandomHorizontalFlip(),
 transforms.ToTensor(),
 transforms.Normalize(mean, std)
]),
 'val': transforms.Compose([
 transforms.Resize(256),
 transforms.CenterCrop(resize),
 transforms.ToTensor(),
 transforms.Normalize(mean, std)
])
 }

 def __call__(self, img, phase):
 return self.data_transform[phase](img)
```

데이터를 가져와서 훈련, 검증, 테스트 용도로 분리

```
cat_directory = r"/content/drive/MyDrive/data/dogs-vs-cats/Cat/"
dog_directory = r"/content/drive/MyDrive/data/dogs-vs-cats/Dog/"

cat_images_filepaths = sorted([os.path.join(cat_directory, f) for f in
 os.listdir(cat_directory)])
dog_images_filepaths = sorted([os.path.join(dog_directory, f) for f in
```

```

os.listdir(dog_directory)])
images_filepaths = [*cat_images_filepaths, *dog_images_filepaths] # 개와 고양이 이미지들을 합쳐서
correct_images_filepaths = [i for i in images_filepaths if cv2.imread(i) is not None]

random.seed(42)
random.shuffle(correct_images_filepaths)
train_images_filepaths = correct_images_filepaths[:400] # 훈련용 400개의 이미지
val_images_filepaths = correct_images_filepaths[400:-10] # 검증용 92개의 이미지
test_images_filepaths = correct_images_filepaths[-10:] # 테스트용 10개의 이미지
print(len(train_images_filepaths), len(val_images_filepaths), len(test_images_filepaths))

```

다음은 훈련, 검증, 테스트에서 사용할 데이터셋의 이미지 수이다.



충분한 데이터가 없으면 과적합이 발생하는 등 테스트 데이터에 대한 성능이 좋지 않다. 성능이 좋은 결과를 원한다면 충분한 데이터셋을 확보하고 테스트를 진행하면 된다. 예를 들어 캐글에서 내려받은 모든 이미지를 사용하는 것뿐 아니라 전처리 부분에서 데이터를 많이 확장시켜 예제를 진행해야 한다.

`torch.utils.data.Dataset`을 상속받아 커스텀 데이터셋을 정의한다.

커스텀 데이터셋 정의

```

class DogvsCatDataset(Dataset):
 def __init__(self, file_list, transform=None, phase='train'):
 self.file_list = file_list # 이미지 데이터가 위치한 파일 경로
 self.transform = transform # 이미지 데이터 전처리
 self.phase = phase # self.phase는 ImageTransform()에서 정의한 'train'과 'val'을 의미

 def __len__(self):
 return len(self.file_list)

 def __getitem__(self, idx):
 img_path = self.file_list[idx] # 이미지 데이터의 인덱스 가져오기
 img = Image.open(img_path)
 img_transformed = self.transform(img, self.phase)

 label = img_path.split('/')[1].split('.')[0] # 레이블 값 가져오기
 if label == 'dog':
 label = 1
 elif label == 'cat':
 label = 0

 return img_transformed, label # 전처리가 적용된 이미지와 레이블 반환

```

전처리에서 필요한 평균, 표준편차 등에 대한 변수 값을 정의한다.

변수에 대한 값 정의

```
size = 256 # AlexNet은 깊이가 깊은 네트워크를 사용하므로 이미지 크기가 256이 아니면 풀링층 때문에 크기가 :
mean = (0.485, 0.456, 0.406)
std = (0.229, 0.224, 0.225)
batch_size = 32
```

훈련과 검증 용도의 데이터셋을 정의한다. 앞에서 정의한 DogvsCatDataset() 클래스에 훈련, 검증, 테스트 데이터를 적용하되 전처리도 함께 적용하도록 한다.

훈련, 검증, 테스트 데이터셋 정의

```
train_dataset = DogvsCatDataset(train_images_filepaths, transform=ImageTransform(size, mean, s
val_dataset = DogvsCatDataset(val_images_filepaths, transform=ImageTransform(size, mean, std),
test_dataset = DogvsCatDataset(val_images_filepaths, transform=ImageTransform(size, mean, std)

index = 0
print(train_dataset.__getitem__(index)[0].size())
print(train_dataset.__getitem__(index)[1])
```

```
torch.Size([3, 256, 256])
0
```

이는 훈련 데이터셋의 크기 및 레이블에 대한 출력 결과이다. 훈련 데이터셋의 크기는 (3,256,256)을 보여 준다. 이것에 대한 값은 (채널, 너비, 높이)를 의미한다.

데이터셋을 데이터로더로 전달하여 메모리로 불러올 준비를 한다.

데이터셋을 메모리로 불러옴

```
train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_dataloader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
test_dataloader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
dataloader_dict = {'train': train_dataloader, 'val': val_dataloader}

batch_iterator = iter(train_dataloader)
inputs, label = next(batch_iterator)
print(inputs.size())
print(label)
```

다음은 데이터로더에서 불러온 훈련 이미지에 대한 크기와 레이블을 출력한 결과이다.



```
torch.Size([32, 3, 256, 256])
tensor([0, 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0,
 0, 1, 0, 1, 0, 0, 1, 1])
```

이제 AlexNet 모델에 대한 네트워크를 정의한다. AlexNet 모델을 사용하기 위한 네트워크는 사전 훈련된 네트워크와 유사하게 정의했다. 합성곱(Conv2d)+(활성화 함수(ReLU))+풀링(MaxPool2d)이 다섯 번 반복된 후 두 개의 완전연결층과 출력층으로 구성된 네트워크이다.

AlexNet 모델 네트워크 정의

```
class AlexNet(nn.Module):
 def __init__(self) -> None:
 super(AlexNet, self).__init__()
 self.features = nn.Sequential(
 nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2),
 nn.ReLU(inplace=True), # ①
 nn.MaxPool2d(kernel_size=3, stride=2),
 nn.Conv2d(64, 192, kernel_size=5, padding=2),
 nn.ReLU(inplace=True),
 nn.MaxPool2d(kernel_size=3, stride=2),
 nn.Conv2d(192, 384, kernel_size=3, padding=1),
 nn.ReLU(inplace=True),
 nn.Conv2d(384, 256, kernel_size=3, padding=1),
 nn.ReLU(inplace=True),
 nn.MaxPool2d(kernel_size=3, stride=2),
)
 self.avgpool = nn.AdaptiveAvgPool2d((6, 6)) # ②
 self.classifier = nn.Sequential(
 nn.Dropout(),
 nn.Linear(256 * 6 * 6, 4096),
 nn.ReLU(inplace=True),
 nn.Dropout(),
 nn.Linear(4096, 512),
 nn.ReLU(inplace=True),
 nn.Linear(512, 2),
)

 def forward(self, x: torch.Tensor) -> torch.Tensor:
 x = self.features(x)
 x = self.avgpool(x)
 x = torch.flatten(x, 1)
 x = self.classifier(x)
 return x
```

① inplace 연산이란 연산에 대한 결과값을 새로운 변수에 저장하는 것이 아닌 기존 데이터를 대체하는 것을 의미한다. 즉, 기존 값을 연산 결과값으로 대체함으로써 기존 값들을 무시하겠다는 의미이다.

② nn.AdaptiveAvgPool2d는 nn.AvgPool2d처럼 풀링을 위해 사용한다. AvgPool2d에서는 풀링에 대한 커널 및 스트라이드 크기를 정의해야 동작한다. 예를 들어, nn.AvgPool2d((3,2), stride=(2,1))

처럼 지정하면 그 결과는 5x5 텐서를 3x3 텐서로, 7x7 텐서를 4x4 텐서로 줄이는 효과를 얻을 수 있다.

$$H_{out} = \left\lceil \frac{H_{in} + 2 \times \text{padding}[0] - \text{kernel\_size}[0]}{\text{stride}[0]} + 1 \right\rceil$$
$$W_{out} = \left\lceil \frac{W_{in} + 2 \times \text{padding}[1] - \text{kernel\_size}[1]}{\text{stride}[1]} + 1 \right\rceil$$

반면에 AdaptiveAvgPool2d는 풀링 작업이 끝날 때 필요한 출력 크기를 정의한다. 즉, nn.AvgPool2d에서는 커널 크기, 스트라이드, 패딩을 지정했다면 nn.AdaptiveAvgPool2d는 출력에 대한 크기만 지정한다. 따라서 AdaptiveAvgPool2d를 사용할 경우 출력 크기 조정이 상당히 쉬워진다. 참고로 AdaptiveAvgPool2d는 입력 크기에 변동이 있고 CNN 위쪽에 완전연결층을 사용하는 경우 유용하다.

앞서 생성한 모델의 네트워크 클래스를 호출하여 model 객체를 생성한다.

model 객체 생성

```
model = AlexNet()
model.to(device)
```



```
AlexNet(
 (features): Sequential(
 (0): Conv2d(3, 64, kernel_size=(11, 11), stride=(4, 4), padding=(2, 2))
 (1): ReLU(inplace=True)
 (2): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
 (3): Conv2d(64, 192, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
 (4): ReLU(inplace=True)
 (5): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
 (6): Conv2d(192, 384, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
 (7): ReLU(inplace=True)
 (8): Conv2d(384, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
 (9): ReLU(inplace=True)
 (10): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
)
 (avgpool): AdaptiveAvgPool2d(output_size=(6, 6))
 (classifier): Sequential(
 (0): Dropout(p=0.5, inplace=False)
 (1): Linear(in_features=9216, out_features=4096, bias=True)
 (2): ReLU(inplace=True)
 (3): Dropout(p=0.5, inplace=False)
 (4): Linear(in_features=4096, out_features=512, bias=True)
 (5): ReLU(inplace=True)
 (6): Linear(in_features=512, out_features=2, bias=True)
)
)
```

학습에서 사용될 옵티마이저와 손실 함수를 정의한다.

옵티마이저 및 손실 함수 정의

```
optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
criterion = nn.CrossEntropyLoss()
```

torchsummary를 이용해 모델의 네트워크를 다시 살펴본다. 모델의 네트워크를 파라미터와 함께 보여 주고 있기 때문에 내부적으로 어떤 일들이 이루어지는지 예측하면서 살펴볼 수 있다.

모델 네트워크 구조 확인

```
from torchsummary import summary
summary(model, input_size=(3,256,256))
```

| Layer (type)                           | Output Shape      | Param #    |
|----------------------------------------|-------------------|------------|
| Conv2d-1                               | [-1, 64, 63, 63]  | 23,296     |
| ReLU-2                                 | [-1, 64, 63, 63]  | 0          |
| MaxPool2d-3                            | [-1, 64, 31, 31]  | 0          |
| Conv2d-4                               | [-1, 192, 31, 31] | 307,392    |
| ReLU-5                                 | [-1, 192, 31, 31] | 0          |
| MaxPool2d-6                            | [-1, 192, 15, 15] | 0          |
| Conv2d-7                               | [-1, 384, 15, 15] | 663,936    |
| ReLU-8                                 | [-1, 384, 15, 15] | 0          |
| Conv2d-9                               | [-1, 256, 15, 15] | 884,992    |
| ReLU-10                                | [-1, 256, 15, 15] | 0          |
| MaxPool2d-11                           | [-1, 256, 7, 7]   | 0          |
| AdaptiveAvgPool2d-12                   | [-1, 256, 6, 6]   | 0          |
| Dropout-13                             | [-1, 9216]        | 0          |
| Linear-14                              | [-1, 4096]        | 37,752,832 |
| ReLU-15                                | [-1, 4096]        | 0          |
| Dropout-16                             | [-1, 4096]        | 0          |
| Linear-17                              | [-1, 512]         | 2,097,664  |
| ReLU-18                                | [-1, 512]         | 0          |
| Linear-19                              | [-1, 2]           | 1,026      |
| Total params: 41,731,138               |                   |            |
| Trainable params: 41,731,138           |                   |            |
| Non-trainable params: 0                |                   |            |
| Input size (MB): 0.75                  |                   |            |
| Forward/backward pass size (MB): 10.03 |                   |            |
| Params size (MB): 159.19               |                   |            |
| Estimated Total Size (MB): 169.97      |                   |            |

모델 학습을 진행할 함수를 정의한다.

모델 학습 함수 정의

```
def train_model(model, dataloader_dict, criterion, optimizer, num_epoch):
 since = time.time()
 best_acc = 0.0

 for epoch in range(num_epoch):
 print('Epoch {}/{}'.format(epoch+1, num_epoch))
 print('-'*20)

 for phase in ['train', 'val']:
 if phase == 'train':
 model.train()
 else:
 model.eval()

 epoch_loss = 0.0
```

```

epoch_corrects = 0

for inputs, labels in tqdm(dataloader_dict[phase]):
 inputs = inputs.to(device)
 labels = labels.to(device)
 optimizer.zero_grad()

 with torch.set_grad_enabled(phase == 'train'):
 outputs = model(inputs)
 _, preds = torch.max(outputs, 1)
 loss = criterion(outputs, labels)

 if phase == 'train':
 loss.backward()
 optimizer.step()

 epoch_loss += loss.item() * inputs.size(0)
 epoch_corrects += torch.sum(preds == labels.data)

epoch_loss = epoch_loss / len(dataloader_dict[phase].dataset)
epoch_acc = epoch_corrects.double() / len(dataloader_dict[phase].dataset)

print('{} Loss: {:.4f} Acc: {:.4f}'.format(phase, epoch_loss, epoch_acc))

if phase == 'val' and epoch_acc > best_acc:
 best_acc = epoch_acc
 best_model_wts = model.state_dict()

time_elapsed = time.time() - since
print('Training complete in {:.0f}m {:.0f}s'.format(
 time_elapsed // 60, time_elapsed % 60))
return model

```

`train_model()` 함수를 호출해 모델을 학습시킨다. 에포크는 10으로 한다.

모델 학습

```

num_epoch = 10
model = train_model(model, dataloader_dict, criterion, optimizer, num_epoch)

```

```

100% ██████████ 13/13 [00:42<00:00, 2.81s/it]
train Loss: 0.6917 Acc: 0.5100
100% ██████████ 3/3 [00:03<00:00, 1.33s/it]
val Loss: 0.6918 Acc: 0.5109
Epoch 9/10

100% ██████████ 13/13 [00:43<00:00, 2.86s/it]
train Loss: 0.6921 Acc: 0.5050
100% ██████████ 3/3 [00:05<00:00, 1.69s/it]
val Loss: 0.6917 Acc: 0.5109
Epoch 10/10

100% ██████████ 13/13 [00:42<00:00, 3.09s/it]
train Loss: 0.6919 Acc: 0.5150
100% ██████████ 3/3 [00:03<00:00, 1.23s/it]
val Loss: 0.6917 Acc: 0.5109
Training complete in 7m 52s

```

모델을 이용한 예측

```

import pandas as pd

id_list = []
pred_list = []
_id = 0
with torch.no_grad():
 for test_path in tqdm(test_images_filepaths): # 테스트 이미지 데이터 이용
 img = Image.open(test_path)
 _id = test_path.split('/')[-1].split('.')[1] # 이미지 데이터의 번호 가져오기 (예를 들어 dog.113)
 transform = ImageTransform(size, mean, std)
 img = transform(img, phase='val') # 테스트 데이터에 검증용 전처리 적용
 img = img.unsqueeze(0)
 img = img.to(device)

 model.eval()
 outputs = model(img)
 preds = F.softmax(outputs, dim=1)[: ,1].tolist()
 id_list.append(_id)
 pred_list.append(preds[0])

res = pd.DataFrame({'id':id_list, 'label':pred_list}) # 데이터 프레임에 이미지 id와 레이블 저장

res.to_csv('/content/drive/MyDrive/data/alexNet.csv', index=False) # 이미지의 id와 레이블을 ale

```

## 데이터 프레임 결과 확인

```
res.head(10)
```

|   | id  | label    |
|---|-----|----------|
| 0 | 145 | 0.511708 |
| 1 | 211 | 0.516832 |
| 2 | 162 | 0.513451 |
| 3 | 200 | 0.520608 |
| 4 | 210 | 0.514900 |
| 5 | 224 | 0.510934 |
| 6 | 213 | 0.512825 |
| 7 | 109 | 0.518171 |
| 8 | 15  | 0.511769 |
| 9 | 167 | 0.510303 |

label이 0.5보다 크면 개, 0.5보다 작으면 고양이

예측 결과를 시각적으로 표현하기 위한 함수 정의

```
class_ = classes = {0:'cat', 1:'dog'}
def display_image_grid(images_filepaths, predicted_labels=(), cols=5):
 rows = len(images_filepaths) // cols
 figure, ax = plt.subplots(nrows=rows, ncols=cols, figsize=(12, 6))
 for i, image_filepath in enumerate(images_filepaths):
 image = cv2.imread(image_filepath)
 image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

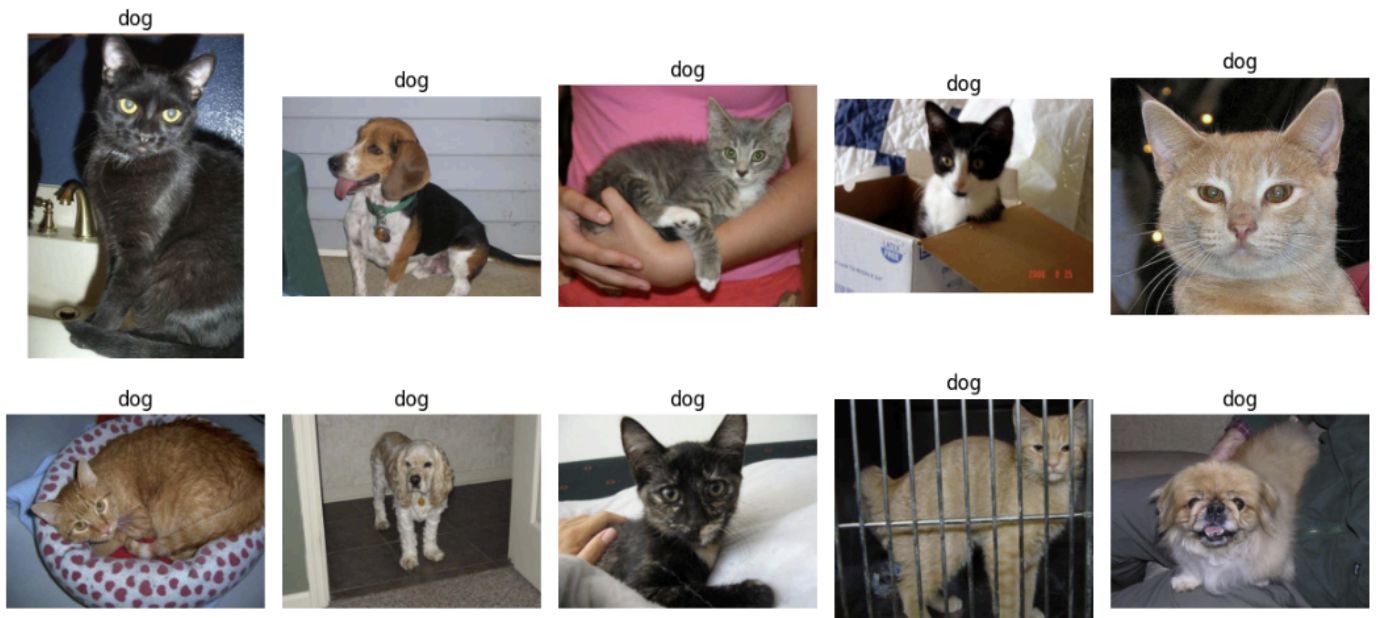
 a = random.choice(res['id'].values)
 label = res.loc[res['id'] == a, 'label'].values[0]
 if label > 0.5:
 label = 1
 else:
 label = 0

 ax.ravel()[i].imshow(image)
 ax.ravel()[i].set_title(class_[label])
 ax.ravel()[i].set_axis_off()
```

```
plt.tight_layout()
plt.show()
```

예측 결과에 대해 이미지와 함께 출력

```
display_image_grid(test_images_filepaths)
```

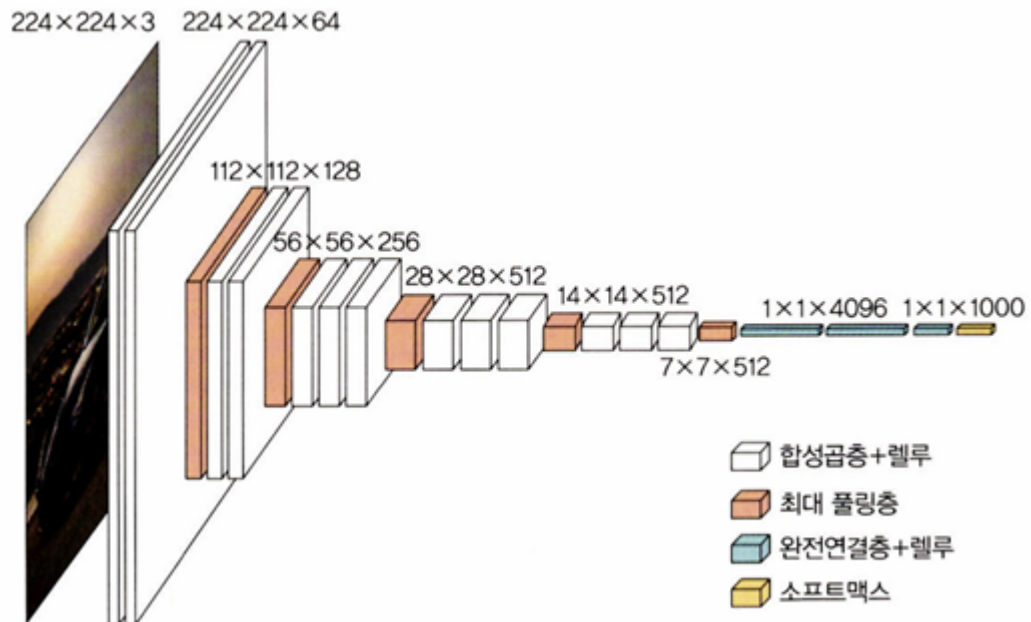


### 6.1.3 VGGNet

VGGNet은 합성곱층의 파라미터 수를 줄이고 훈련 시간을 개선하고자 탄생했다. 즉, 네트워크를 깊게 만드는 것이 성능에 어떤 영향을 미치는지 확인하고자 나왔다. VGG 연구 팀은 깊이의 영향만 최대한 확인하고자 합성곱층에서 사용하는 필터/커널의 크기를 가장 작은 3x3으로 고정했다.

네트워크 계층의 총 개수에 따라 여러 유형의 VGGNet(VGG16,VGG19)이 있다.

VGG16에는 파라미터가 총 1억 3300만 개 있다. 모든 합성곱 커널의 크기는 3x3, 최대 풀링 커널의 크기는 2x2이며, 스트라이드는 2다. 결과적으로 64개의 224x224 특성 맵(224x224x64)들이 생성된다. 그리고 마지막 16번째 계층을 제외하고는 모두 ReLU 활성화 함수가 적용된다.



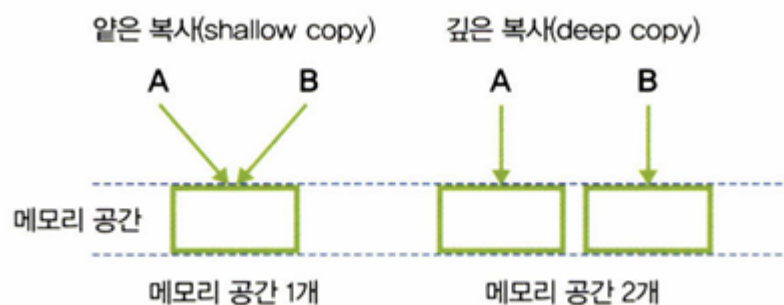
이 예제에서는 VGGNet 중 가장 간단한 VGG11을 구현한다.

필요한 라이브러리 호출

```
import copy # ①
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torch.utils.data as data
import torchvision
import torchvision.transforms as transforms
import torchvision.datasets as Datasets

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

① 객체 복사를 위해 사용한다. 객체 복사는 크게 얇은 복사와 깊은 복사로 나뉜다.



## VGG 모델 정의

```
class VGG(nn.Module):
 def __init__(self, features, output_dim):
 super().__init__()
 self.features = features # VGG 모델에 대한 매개변수에서 받아 온 features 값을 self.features에 넣
 self.avgpool = nn.AdaptiveAvgPool2d(7)
 self.classifier = nn.Sequential(
 nn.Linear(512*7*7, 4096),
 nn.ReLU(inplace=True),
 nn.Dropout(0.5),
 nn.Linear(4096, 4096),
 nn.ReLU(inplace=True),
 nn.Dropout(0.5),
 nn.Linear(4096, output_dim)
) # 완전연결층과 출력층 정의

 def forward(self, x):
 x = self.features(x)
 x = self.avgpool(x)
 h = x.view(x.shape[0], -1)
 x = self.classifier(h)
 return x, h
```

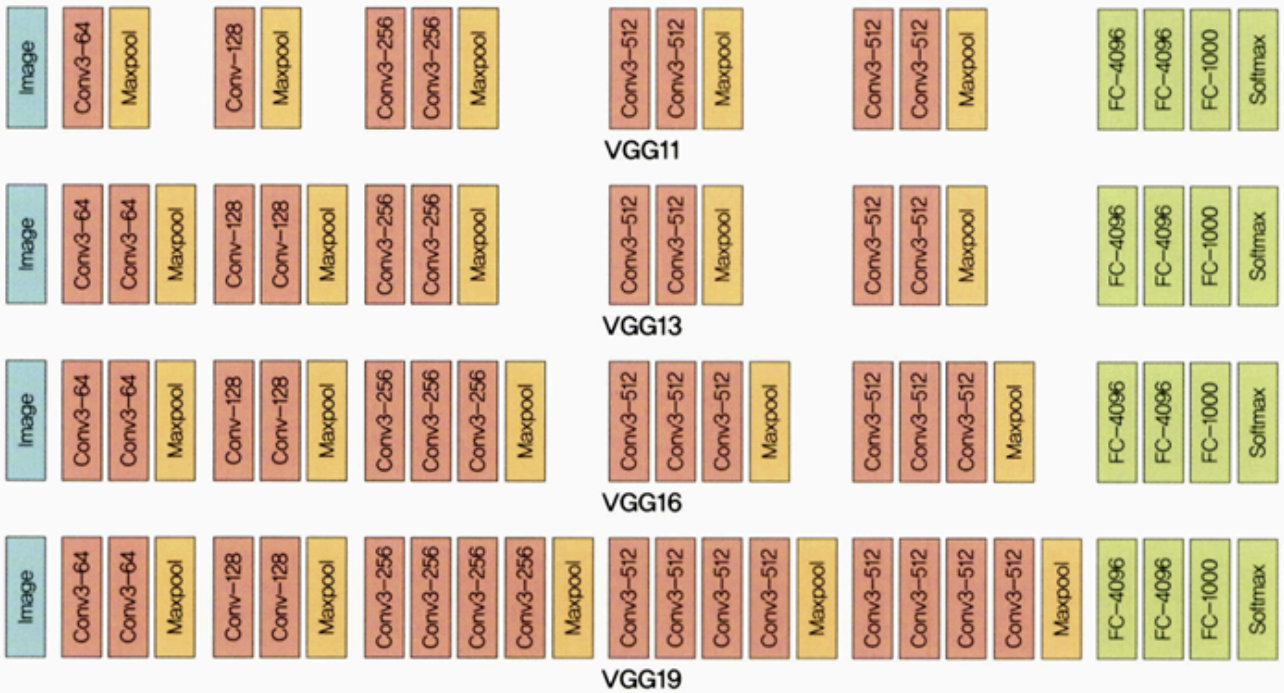
다음 코드는 VGG11, VGG13, VGG16, VGG19 모델의 계층을 정리한 것이다. 숫자(output channel, 출력 채널)는 Conv2d를 수행하라는 의미이며, 출력 채널이 다음 계층의 입력 채널이 된다. 또한, M은 최대 풀링을 수행하라는 의미이다.

## 모델 유형 정의

```
vgg11_config = [64, 'M', 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M'] # 8(합성곱층)+3
vgg13_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M'] # 10(
vgg16_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 'M', 512, 512, 512, 'M', 512, 512,
vgg19_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 256, 'M', 512, 512, 512, 512, 'M',
```



▼ 그림 6-15 VGG의 다양한 모델에 대한 네트워크



VGG 계층 정의

```
def get_vgg_layers(config, batch_norm):
 layers = []
 in_channels = 3

 for c in config : # vgg11_config 값들을 가져옴
 assert c == 'M' or isinstance(c, int) # ①
 if c == 'M': # 불러온 값이 'M'이면 최대 풀링 적용
 layers += [nn.MaxPool2d(kernel_size = 2)]
 else: # 불러온 값이 숫자이면 합성곱(Conv2d) 적용
 conv2d = nn.Conv2d(in_channels, c, kernel_size=3, padding=1)
 if batch_norm: # 배치 정규화를 적용할지에 대한 코드
 layers += [conv2d, nn.BatchNorm2d(c), nn.ReLU(inplace=True)] # 배치 정규화가 적용될 경우 t
 else:
 layers += [conv2d, nn.ReLU(inplace=True)] # 배치 정규화가 적용되지 않을 경우 ReLU만 적용
 in_channels = c

 return nn.Sequential(*layers) # 네트워크의 모든 계층 반환
```

① 조건문을 정의한다.

assert c == 'M' or isinstance(c, int)

(a)

(b)

- ㉑ assert는 뒤의 조건이 True가 아니면 에러를 발생시킨다. 따라서 `c == 'M'`이 아니면 오류가 발생한다.
- ㉒ isinstance는 주어진 조건이 True인지 판단한다.

```
print(isinstance(1, int)) ----- 1이 integer인지 판단
print(isinstance(1.2, int)) ----- 1.2가 integer인지 판단
print(isinstance('deep learning', str)) ----- deep learning이 string인지 판단
```

`get_vgg_layers()` 함수를 호출해 모델의 계층을 생성한다. 이때 배치 정규화에 대한 계층도 추가한다.

모델 계층 생성

```
vgg11_layers = get_vgg_layers(vgg11_config, batch_norm=True) # ㉑
```

㉑ `batch_norm`(Batch Normalization)은 데이터의 평균을 0, 표준편차를 1로 분포시키는 것이다. 각 계층에서 입력 데이터의 분포는 앞 계층에서 업데이트된 가중치에 따라 변한다. 즉, 각 계층마다 변화되는 분포는 학습 속도를 늦출 뿐 아니라 학습도 어렵게 한다. 따라서 각 계층의 입력에 대한 분산을 평균 0, 표준편차 1로 분포시키는 것이 `batch_norm`이다.

VGG11 계층 확인

```
print(vgg11_layers)
```

```

Sequential(
 (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
 (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (2): ReLU(inplace=True)
 (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
 (4): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
 (5): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (6): ReLU(inplace=True)
 (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
 (8): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
 (9): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (10): ReLU(inplace=True)
 (11): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
 (12): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (13): ReLU(inplace=True)
 (14): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
 (15): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
 (16): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (17): ReLU(inplace=True)
 (18): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
 (19): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (20): ReLU(inplace=True)
 (21): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
 (22): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
 (23): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (24): ReLU(inplace=True)
 (25): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
 (26): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (27): ReLU(inplace=True)
 (28): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
)

```

VGG11 전체에 대한 네트워크

```

OUTPUT_DIM = 2 # 개와 고양이 두 클래스 사용
model = VGG(vgg11_layers, OUTPUT_DIM)
print(model)

```

```

VGG(
 (features): Sequential(
 (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
 (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (2): ReLU(inplace=True)
 (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
 (4): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
 (5): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (6): ReLU(inplace=True)
 (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
 (8): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
 (9): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (10): ReLU(inplace=True)
 (11): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
 (12): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (13): ReLU(inplace=True)
 (14): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
 (15): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
 (16): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (17): ReLU(inplace=True)
 (18): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
 (19): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (20): ReLU(inplace=True)
 (21): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
 (22): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
 (23): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (24): ReLU(inplace=True)
 (25): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
 (26): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (27): ReLU(inplace=True)
 (28): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
)
 (avgpool): AdaptiveAvgPool2d(output_size=7)
 (classifier): Sequential(
 (0): Linear(in_features=25088, out_features=4096, bias=True)
 (1): ReLU(inplace=True)
 (2): Dropout(p=0.5, inplace=False)
 (3): Linear(in_features=4096, out_features=4096, bias=True)
 (4): ReLU(inplace=True)
 (5): Dropout(p=0.5, inplace=False)
 (6): Linear(in_features=4096, out_features=2, bias=True)
)
)

```

## VGG11 사전 훈련된 모델 사용

```

import torchvision.models as models
pretrained_model = models.vgg11_bn(pretrained=True) # ①
print(pretrained_model)

```

① 배치 정규화가 적용된 사전 훈련된 VGG11 모델을 사용하기 위해서는 다음과 같은 파라미터를 사용한다.

```
pretrained_model = models.vgg11_bn(pretrained=True)
```

Ⓐ

Ⓑ

Ⓐ vgg11\_bn은 VGG11 기본 모델에 배치 정규화가 적용된 모델을 사용하겠다는 의미이다.

Ⓑ pretrained를 True로 설정하면 사전 훈련된 모델을 사용(미리 학습된 파라미터 값들을 사용)하겠다는 의미이다.

```
Warning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.
Warning: Arguments other than a weight enum or 'None' for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing 'weights=VGG11_BN_Weights.IMAGENET1001'.
Downloading: "https://download.pytorch.org/models/vgg11_bn-6022393d.pth" to /root/.cache/torch/hub/checkpoints/vgg11_bn-6022393d.pth
100% 507M/507M [00:07<00:00, 74.3MB/s]
VGG(
 (features): Sequential(
 (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
 (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (2): ReLU(inplace=True)
 (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
 (4): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
 (5): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (6): ReLU(inplace=True)
 (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
 (8): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
 (9): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (10): ReLU(inplace=True)
 (11): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
 (12): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (13): ReLU(inplace=True)
 (14): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
 (15): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
 (16): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (17): ReLU(inplace=True)
 (18): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
 (19): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (20): ReLU(inplace=True)
 (21): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
 (22): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
 (23): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (24): ReLU(inplace=True)
 (25): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
 (26): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
 (27): ReLU(inplace=True)
 (28): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
)
 (avgpool): AdaptiveAvgPool2d(output_size=(7, 7))
 (classifier): Sequential(
 (0): Linear(in_features=25088, out_features=4096, bias=True)
 (1): ReLU(inplace=True)
 (2): Dropout(p=0.5, inplace=False)
 (3): Linear(in_features=4096, out_features=4096, bias=True)
 (4): ReLU(inplace=True)
 (5): Dropout(p=0.5, inplace=False)
 (6): Linear(in_features=4096, out_features=1000, bias=True)
)
)
```

이미지 데이터 전처리

```
train_transforms = transforms.Compose([
 transforms.Resize((256, 256)),
 transforms.RandomRotation(5),
 transforms.RandomHorizontalFlip(0.5),
 transforms.ToTensor(),
 transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) # ㉠
])
```

```
test_transforms = transforms.Compose([
 transforms.Resize((256, 256)),
 transforms.ToTensor(),
 transforms.Normalize(mean=[0.485, 0.456, 0.456], std=[0.229, 0.224, 0.225])
])
```

```
transforms.Compose([transforms.Resize((256, 256)), transforms.RandomRotation(5),])
```

㉑

㉒

㉑ transforms.Resize: 이미지를 주어진 크기로 재조정한다. 즉, 256x256 크기로 이미지를 조정한다.

㉒ transforms.RandomRotation: 5도 이하로 이미지를 회전시킨다.

ImageFolder를 이용하여 데이터셋 불러오기

```
train_path = "/content/drive/MyDrive/data/catanddog/train" # 훈련 데이터셋 경로
test_path = "/content/drive/MyDrive/data/catanddog/test" # 테스트 데이터셋 경로
```

```
train_dataset = torchvision.datasets.ImageFolder(
 train_path,
 transform=train_transforms
) # ㉑
```

```
test_dataset = torchvision.datasets.ImageFolder(
 test_path,
 transform=test_transforms
)
```

```
print(len(train_dataset))
print(len(test_dataset))
```

㉑ ImageFolder는 계층적인 폴더 구조를 갖고 있는 데이터셋을 불러올 때 사용한다.

538  
12

훈련과 검증 데이터 분할

```
VALID_RATIO = 0.9
n_train_examples = int(len(train_dataset) * VALID_RATIO) # 전체 훈련 데이터 중 90%를 훈련 데이터셋으로
n_valid_examples = len(train_dataset) - n_train_examples # 전체 훈련 데이터 중 10%를 검증 데이터셋으로
train_data, valid_data = data.random_split(train_dataset, [n_train_examples, n_valid_examples])
```

㉑ random\_split()은 훈련과 검증 데이터셋을 나누는 용도로 사용한다. 데이터가 데이터로더로 넘어간 이후에는 분리가 불가능하므로 데이터셋 단계에서 진행해야 한다.

```
train_data, valid_data = data.random_split(train_dataset,
 (a)
 [n_train_examples, n_valid_examples])
 (b)
```

- ㉠ 첫 번째 파라미터: 분할에 사용될 데이터셋
- ㉡ 두 번째 파라미터: 훈련과 검증 데이터셋의 크기 지정. [훈련 데이터셋 크기, 검증 데이터셋 크기]

검증 데이터 전처리

```
valid_data = copy.deepcopy(valid_data)
valid_data.dataset.transform = test_transformsv
```

## 훈련, 검증, 테스트 데이터셋 수 확인

```
print(f'Number of training examples: {len(train_data)}')
print(f'Number of validation examples: {len(valid_data)}')
print(f'Number of testing examples: {len(test_dataset)}')
```

```
Number of training examples: 484
Number of validation examples: 54
Number of testing examples: 12
```

데이터로더를 이용해 데이터셋의 데이터를 메모리로 가져온다. 데이터를 가져올 때는 배치 크기만큼 나누어서 가져온다.

메모리로 데이터 불러오기

```
BATCH_SIZE = 128
train_iterator = data.DataLoader(train_data,
 shuffle=True,
 batch_size=BATCH_SIZE) # 훈련 데이터셋은 임의로 섞어서 가져옴

valid_iterator = data.DataLoader(valid_data,
 batch_size=BATCH_SIZE)

test_iterator = data.DataLoader(test_dataset,
 batch_size=BATCH_SIZE)
```

## 옵티마이저와 손실 함수 정의

```
optimizer = optim.Adam(model.parameters(), lr=1e-7)
criterion = nn.CrossEntropyLoss()

model = model.to(device)
criterion = criterion.to(device)
```

## 모델 정확도 측정 함수

```
def calculate_accuracy(y_pred, y):
 top_pred = y_pred.argmax(1, keepdim=True)
 correct = top_pred.eq(y.view_as(top_pred)).sum() # ①
 acc = correct.float() / y.shape[0]
 return acc
```

① 예측이 정답과 일치하는 경우 그 개수의 합을 correct 변수에 저장한다.

correct = top\_pred.eq(y.view\_as(top\_pred)).sum()

(a)                      (b)                      (c)

① eq는 equal의 약자로 서로 같은지를 비교하는 표현식이다.

② view\_as(other)는 other의 텐서 크기를 사용하겠다는 의미이다. 즉, y.view\_as(top\_pred)는 y에 대한 텐서 크기를 top\_pred의 텐서 크기로 변경하겠다는 의미이다.

③ 합계를 구하는 것으로, 여기에서는 예측과 정답이 일치하는 것들의 개수를 합산하겠다는 의미이다.

## 모델 학습 함수 정의

```
def train(model, iterator, optimizer, criterion, device):
 epoch_loss = 0
 epoch_acc = 0

 model.train()
 for (x,y) in iterator:
 x = x.to(device)
 y = y.to(device)

 optimizer.zero_grad()
 y_pred, _ = model(x)
 loss = criterion(y_pred, y)
 acc = calculate_accuracy(y_pred, y)
 loss.backward()
 optimizer.step()
```



```

epoch_loss += loss.item()
epoch_acc += acc.item()

return epoch_loss / len(iterator), epoch_acc / len(iterator)

```

모델 성능 측정 함수

```

def evaluate(model, iterator, criterion, device):
 epoch_loss = 0
 epoch_acc = 0

 model.eval()
 with torch.no_grad():
 for (x,y) in iterator:
 x = x.to(device)
 y = y.to(device)
 y_pred, _ = model(x)
 loss = criterion(y_pred, y)
 acc = calculate_accuracy(y_pred, y)

 epoch_loss += loss.item()
 epoch_acc += acc.item()

 return epoch_loss / len(iterator), epoch_acc / len(iterator)

```

학습 시간 측정 함수

```

def epoch_time(start_time, end_time):
 elapsed_time = end_time - start_time
 elapsed_mins = int(elapsed_time / 60)
 elapsed_secs = int(elapsed_time - (elapsed_mins * 60))
 return elapsed_mins, elapsed_secs

```

모델 학습

```

import time

EPOCHS = 5
best_valid_loss = float('inf')
for epoch in range(EPOCHS):
 start_time = time.monotonic()
 train_loss, train_acc = train(model, train_iterator, optimizer, criterion, device) # 훈련 데이터
 valid_loss, valid_acc = evaluate(model, valid_iterator, criterion, device) # 검증 데이터셋을 도

 if valid_loss < best_valid_loss: # valid_loss가 가장 작은 값을 구하고 그 상태의 모델을 VGG-model.pt로 저장
 best_valid_loss = valid_loss
 torch.save(model.state_dict(), '/content/drive/MyDrive/data/VGG-model.pt')

 end_time = time.monotonic()
 epoch_mins, epoch_secs = epoch_time(start_time, end_time) # 모델 훈련에 대한 시작과 종료 시간을 기록

 print(f'Epoch: {epoch+1:02} | Epoch Time: {epoch_mins}m {epoch_secs}s')

```

```
print(f'\tTrain Loss: {train_loss:.3f} | Train Acc: {train_acc*100:.2f}%')
print(f'\t Valid. Loss: {valid_loss:.3f} | Valid. Acc: {valid_acc*100:.2f}%')
```

```
Epoch: 01 | Epoch Time: 1m 1s
 Train Loss: 0.696 | Train Acc: 52.62%
 Valid. Loss: 0.694 | Valid. Acc: 50.52%
Epoch: 02 | Epoch Time: 0m 14s
 Train Loss: 0.693 | Train Acc: 50.40%
 Valid. Loss: 0.692 | Valid. Acc: 52.08%
Epoch: 03 | Epoch Time: 0m 18s
 Train Loss: 0.695 | Train Acc: 48.99%
 Valid. Loss: 0.689 | Valid. Acc: 51.04%
Epoch: 04 | Epoch Time: 0m 15s
 Train Loss: 0.694 | Train Acc: 50.60%
 Valid. Loss: 0.687 | Valid. Acc: 55.21%
Epoch: 05 | Epoch Time: 0m 14s
 Train Loss: 0.697 | Train Acc: 52.82%
 Valid. Loss: 0.684 | Valid. Acc: 62.50%
```

테스트 데이터셋을 이용한 모델 성능 측정

```
model.load_state_dict(torch.load('/content/drive/MyDrive/data/VGG-model.pt'))
test_loss, test_acc = evaluate(model, test_iterator, criterion, device)
print(f'Test Loss: {test_loss:.3f} | Test Acc: {test_acc*100:.2f}%')
```

```
Test Loss: 0.672 | Test Acc: 66.67%
```

테스트 데이터셋을 이용한 모델의 예측 확인 함수

```
def get_predictions(model, iterator):
 model.eval()
 images = []
 labels = []
 probs = []

 with torch.no_grad():
 for (x,y) in iterator:
 x = x.to(device)
 y_pred, _ = model(x)
 y_prob = F.softmax(y_pred, dim=-1)
```

```

top_pred = y_prob.argmax(1, keepdim=True) # ①
images.append(x.cpu())
labels.append(y.cpu())
probs.append(y_prob.cpu())

images = torch.cat(images, dim=0) # ②
labels = torch.cat(labels, dim=0)
probs = torch.cat(probs, dim=0)
return images, labels, probs

```

① argmax는 배열에서 가장 큰 값의 인덱스를 찾을 때 사용한다.

```

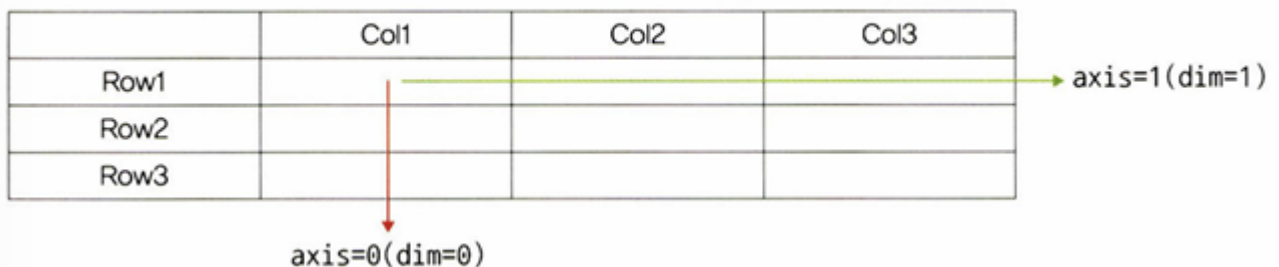
top_pred = y_prob.argmax(1, keepdim=True)
 (a) (b)

```

① 첫 번째 파라미터: 행(axis=0) 또는 열(axis=1)을 따라 가장 큰 값의 인덱스를 찾는다. 따라서 1은 열을 따라 가장 큰 값의 인덱스를 찾겠다는 의미이다.

② torch.cat은 텐서를 연결할 때 사용한다.

▼ 그림 6-17 차원(dim)



예측 중에서 정확하게 예측한 것을 추출

```

images, labels, probs = get_predictions(model, test_iterator)
pred_labels = torch.argmax(probs, 1) # ①
corrects = torch.eq(labels, pred_labels) # 예측과 정답이 같은지 비교
correct_examples = []

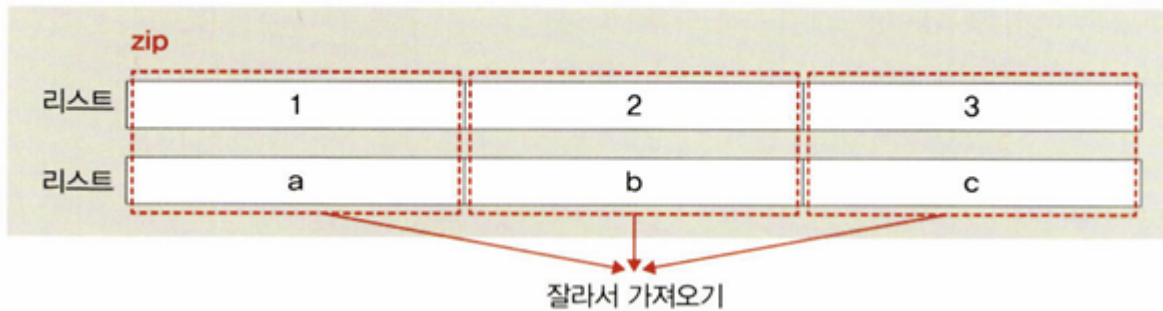
for image, label, prob, correct in zip(images, labels, probs, corrects): # ②
 if correct:
 correct_examples.append((image, label, prob))

correct_examples.sort(reverse=True, key=lambda x: torch.max(x[2], dim=0).values) # ③

```

- ① max는 최댓값을 반환하고, argmax는 최댓값을 갖는 인덱스를 반환한다.
- ② zip()은 여러 개의 리스트(혹은 튜플)을 합쳐서 새로운 튜플 타입으로 반환한다.

▼ 그림 6-18 zip()의 원리



- ③ 데이터를 정렬하기 위해 sort() 메서드를 사용하며, 파라미터는 다음과 같다.

```
correct_examples.sort(reverse=True, key=lambda x: torch.max(x[2], dim=0).values)
```

(a) (b)

- ① reverse: 내림차순으로 정렬한다.
- ② key: 데이터를 정렬할 때 key 값을 가지고 정렬하며 기본값은 오름차순이다. 람다는 일종의 함수로, 함수명 없이도 사용 가능하다.

▼ 그림 6-19 일반 함수와 람다 함수

```
def 함수명(매개변수):
 return 반환될 결과값

lamda 매개변수 : 반환될 결과값
```

람다 함수는 정의와 동시에 사용할 수 있지만 함수명이 없고, 저장된 변수가 없기 때문에 재사용이 불가능하다.

## 이미지 출력을 위한 전처리

```
def normalize_image(image):
 image_min = image.min()
 image_max = image.max()
 image.clamp_(min=image_min, max=image_max) # torch.clamp는 주어진 최소, 최대의 범주에 이미지가 위치
 image.add_(-image_min).div_(image_max - image_min + 1e-5) # ①
 return image
```

① torch.add는 말 그대로 더하기 메서드이다.

torch.add에서는 새로운 메모리 공간이 할당되어 저장되기 때문에 x와 y가 같은지 물었을 때 False를 출력하는 반면, torch.add\_는 새로운 공간 할당 없이 기존의 메모리에 위치한 값을 대체한다.

모델이 정확하게 예측한 이미지 출력 함수

```
def plot_most_correct(correct, classes, n_images, normalize=True):
 rows = int(np.sqrt(n_images)) # np.sqrt는 제곱근을 계산(0.5를 거듭제곱)
 cols = int(np.sqrt(n_images))
 fig = plt.figure(figsize=(25,20))
 for i in range(rows*cols):
 ax = fig.add_subplot(rows, cols, i+1) # 출력하려는 그래프 개수만큼 subplot을 만들
 image, true_label, probs = correct[i]
 image = image.permute(1, 2, 0) # ①
 true_prob = probs[true_label]
 correct_prob, correct_label = torch.max(probs, dim=0)
 true_class = classes[true_label]
 correct_class = classes[correct_label]

 if normalize: # 본래 이미지대로 출력하기 위해 normalize_image 함수 호출
 image = normalize_image(image)

 ax.imshow(image.cpu().numpy())
 ax.set_title(f'true label: {true_class} ({true_prob:.3f})\n' \
 f'pred label: {correct_class} ({correct_prob:.3f})')
 ax.axis('off')

 fig.subplots_adjust(hspace=0.4)
```

① image.permute는 축을 변경할 때 사용한다.

예측 결과 이미지 출력

```
import matplotlib.pyplot as plt

classes = test_dataset.classes
N_IMAGES = 5
plot_most_correct(correct_examples, classes, N_IMAGES)
```

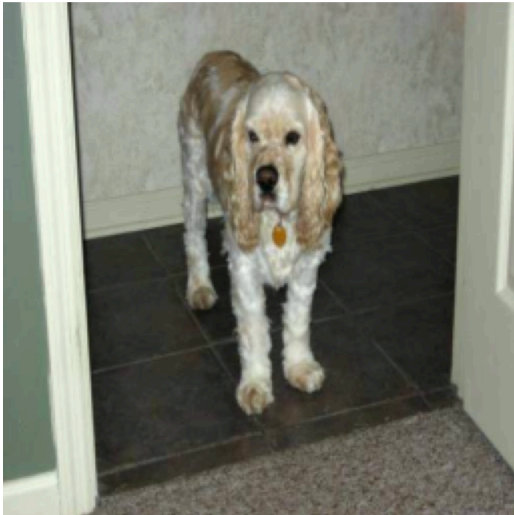
true label: Dog (0.535)  
pred label: Dog (0.535)



true label: Dog (0.532)  
pred label: Dog (0.532)



true label: Dog (0.513)  
pred label: Dog (0.513)



true label: Cat (0.507)  
pred label: Cat (0.507)



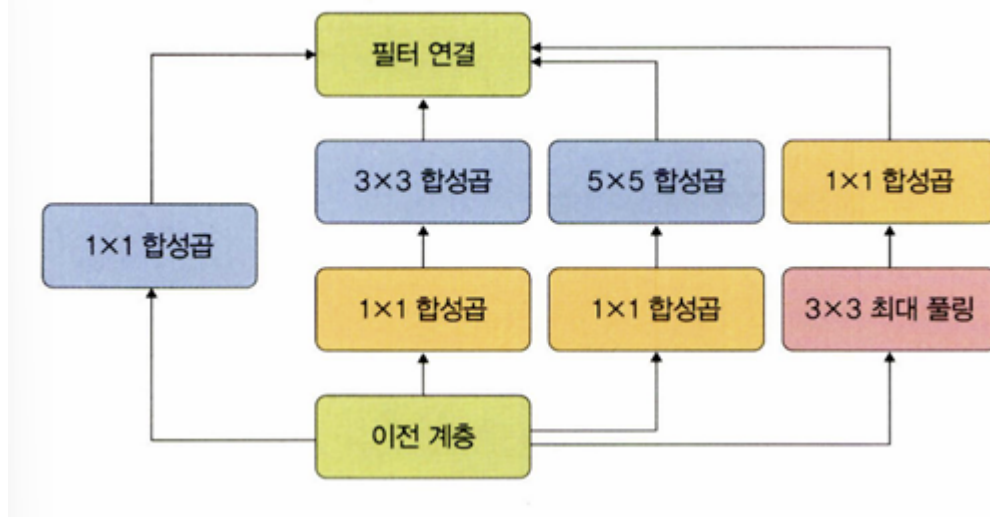
#### 6.1.4 GoogLeNet

GoogLeNet은 주어진 하드웨어 자원을 최대한 효율적으로 이용하면서 학습 능력은 극대화할 수 있는 깊고 넓은 신경망이다.

깊고 넓은 신경망을 위해 GoogLeNet은 인셉션 모듈을 추가했다. 인셉션 모듈에서는 특징을 효율적으로 추출하기 위해 1x1, 3x3, 5x5의 합성곱 연산을 각각 수행한다. 3x3 최대 풀링은 입력과 출력의 높이와 너비가 같아야 하므로 패딩을 추가해야 한다. 결과적으로 GoogLeNet에 적용된 해결 방법은 희소 연결이다.

CNN은 합성곱, 풀링, 완전연결층이 서로 밀집하게 연결되어 있다. 뻘뻘하게 연결된 신경망 대신 관련성이 높은 노드끼리만 연결하는 방법을 희소 연결이라고 한다. 이것으로 연산량이 적어지며 과적합도 해결할 수 있다.

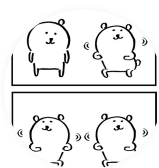
▼ 그림 6-23 GoogLeNet의 인셉션 모듈



인셉션 모듈의 네 가지 연산은 다음과 같다.

- 1×1 합성곱
- 1×1 합성곱 + 3×3 합성곱
- 1×1 합성곱 + 5×5 합성곱
- 3×3 최대 풀링(maxpooling) + 1×1 합성곱(convolutional)

딥러닝을 이용해 ImageNet과 같은 대회에 참여하거나 서비스를 제공하려면 대용량 데이터를 학습해야 한다. 심층 신경망의 아키텍처에서 계층이 넓고(뉴런이 많고) 깊으면(계층이 많으면) 인식률은 좋아지지만, 과적합이나 기울기 소멸 문제를 비롯한 학습 시간 지연과 연산 속도 증가 등의 문제가 있다. 특히 합성곱 신경망에서 이런 문제들이 자주 나타나는데, GoogLeNet으로 이러한 문제를 해결할 수 있다.



**카사바**

안녕하세요 :)



이전 포스트

Week4\_예습과제

## 0개의 댓글

댓글을 작성하세요

댓글 작성



Powered by  
**Stellate**